

Adversarial Independent Security Review — Security Response Headers (commit 992bd497)

Revision: 1 **Last modified:** 2026-07-10T00:00:00Z

Scope: `server/internal/api/security_headers.go`, `security_headers_test.go`, `server.go` (middleware order), `embed.go` (Tier-C SPA CSP). Read-only adversarial review per §11.4.125/§11.4.134/§11.4.142. Verdict at the bottom.

Every claim below was checked against real evidence — the actual gin v1.12.0 vendored source (`/home/milos/go/pkg/mod/github.com/gin-gonic/gin@v1.12.0`), the real go test run of the 8 tests, and a byte-level grep of the ACTUAL embedded SPA bundle (`server/internal/api/manager-dist/assets/index-*.js`) — not assumptions. No finding here is speculative.

Verdict: GO-WITH-FIXES

Severity	Count
Critical	1
Important	3
Minor	2
Nit	1

The Tier-A/B header middleware itself (`security_headers.go`) is well-designed and correctly wired (confirmed via gin internals, not just the tests). The **one Critical finding is not in `security_headers.go` at all** — it is a build-artifact defect in the embedded SPA that silently invalidates the `connect-src 'self'` directive's stated justification and will break the Manager UI outside of a coincidental `localhost:8080 dev`

topology. It must be fixed (or at minimum tracked + the CSP evidence doc corrected) before this is considered done, hence GO-WITH-FIXES rather than a clean GO.

CRITICAL

C1 — connect-src 'self' justification is false for the actual shipped SPA bundle; Manager UI will be non-functional (or CSP-blocked) in any real deployment

Files: server/internal/api/security_headers.go:63-65 (the connect-src comment) · server/internal/api/embed.go:6-16 (undocumented build step) · clients/ota-manager/src/lib/api-client.ts:11-17 · shipped artifact server/internal/api/manager-dist/assets/index-{C51iU0QH,BCUfw0_g}.js

Failure scenario: The CSP's connect-src 'self' comment claims: *“the SPA calls its own API same-origin (axios baseURL is the serving origin in the single-binary topology).”* This is **false for the actual embedded bundle**. Byte-level inspection of the real, currently-embedded JS confirms:

```
h4="http://
localhost:8080",ty=Nt.create({baseURL:h4,timeout:3e4,...})
```

i.e. axios's baseURL is a **compile-time, hardcoded, absolute** URL literal (http://localhost:8080) baked into the bundle at pnpm build time from clients/ota-manager/src/lib/api-client.ts:14: const BASE_URL = import.meta.env.VITE_API_BASE_URL ?? "http://localhost:8080".

clients/ota-manager/README.md:59 and the standalone-nginx docker/ota-manager.docker-compose.yml:35-38 both document that VITE_API_BASE_URL=/api is the value required for the same-origin (single-binary or nginx-proxied) topology — **but embed.go's own documented “Usage” build steps (cd clients/ota-manager && pnpm build) never set this env var**, and no build/CI script anywhere in the repo sets it for the go:embed path either (verified: repo-wide grep for VITE_API_BASE_URL outside an unrelated submodules/helixqa vendor script finds nothing). The currently-embedded artifact was built exactly this way and ships the hardcoded http://localhost:8080 value.

Consequence: 'self' in CSP is evaluated by the browser against the **actual page origin at load time** (whatever host/port/scheme the operator really deployed at), not against a string baked into the JS at build time. In literally any deployment that is not exactly `http://localhost:8080` — any real production host/port, any TLS deployment, anything behind a reverse proxy — the SPA's own `fetch/XHR` calls target a different origin than the page, so the browser's CSP enforcement of `connect-src 'self'` **blocks every API call the Manager UI makes**: login, device list, everything. (If CSP somehow didn't block it, an HTTPS deployment would separately hit mixed-content blocking for the same reason.) It “works” today only by coincidence, because dev/test happens to run at `:8080`.

The project's own evidence doc bakes in the same error: `docs/qa/20260710-server-security-headers/EVIDENCE.md:119` states “*axios baseURL="http://localhost:8080" = the serving origin in the single-binary topology ... 'self' covers same-origin API calls*” — this conflates a dev-machine coincidence with what 'self' actually means, and none of the 8 Go tests can catch this class of defect (it requires a real browser enforcing CSP against a real `fetch` call; `httptest` never drives a browser or inspects the compiled bundle's network target).

Fix: 1. Set `VITE_API_BASE_URL=/api` (or the correct relative value — confirm `axios`'s behavior for an empty/relative `baseURL` is same-origin, not undefined) before `pnpm build` for the `go:embed` deployment path, and update `embed.go`'s documented “Usage” build steps (lines 6-16) to state this explicitly, mirroring the `docker-compose` comment. 2. Regenerate `manager-dist/` with the corrected env var. 3. Add a build-time or CI assertion that greps the built bundle for the literal `http://localhost/` any absolute `http://https://` `axios baseURL` and fails the build if found — so this cannot silently regress (this is exactly the class of defect §11.4.108 “source vs. artifact” and §11.4.110 “change-impact clash detection” exist to catch mechanically). 4. Correct `EVIDENCE.md`'s `connect-src` row to state the actual requirement (relative/same-origin `baseURL`) instead of rationalizing the coincidental dev value as if it were architecturally guaranteed.

IMPORTANT

I1 — HSTS (and Tier-C upgrade-insecure-requests) gate on “this process terminates TLS,” not on “the browser sees HTTPS” — silently absent behind a TLS-terminating reverse proxy

File: server/internal/api/security_headers.go:124-126 (tlsEnabled())
· server/internal/config/config.go:79-108

tlsEnabled() is `s.cfg.TLSCertFile != "" && s.cfg.TLSKeyFile != ""` — i.e. purely whether *this Go process* holds a cert/key pair. There is no X-Forwarded-Proto / trusted-proxy detection and no explicit config override (confirmed: no such handling anywhere in config.go or internal/api/*.go). In the very common production topology where TLS is terminated at an edge (k8s ingress, ALB, Cloudflare, nginx — plausible given this project’s containerized rollout per CLAUDE.md) and the Go binary is deployed serving plain HTTP internally, HELIX_TLS_CERT/HELIX_TLS_KEY are unset, so tlsEnabled() returns false and **HSTS is never emitted even though real end users are on HTTPS** — silently dropping the primary browser defense against SSL-stripping/downgrade attacks on the browser-to-edge leg, for what is likely the *most common* real deployment shape, not an edge case.

Fix: add an explicit config override (e.g. HELIX_BEHIND_TLS_PROXY=true) or honor a trusted X-Forwarded-Proto: https from a documented/allow-listed proxy, so operators terminating TLS upstream still get HSTS + the CSP upgrade-insecure-requests directive.

I2 — The 429-shed “headers survive” test does not exercise the real production middleware chain; a real ordering regression would ship undetected

File: server/internal/api/security_headers_test.go:107-141
(TestSecurityHeaders_TierA_OnShed429)

The test file’s own header comment (lines 20-23) states these tests “exercise the REAL wired router ... not a hand-rolled stand-in, so a PASS proves the headers reach real responses.”

TestSecurityHeaders_TierA_OnShed429 is the **one exception**: it builds its own bespoke router — `r := gin.New();`
`r.Use(securityHeadersMiddleware(false), maxInflightMiddleware(1))`
— hard-coding the correct order, rather than calling the real

`Server.Router()` (`server.go:122-123`). Verified: `newTestEnv` (`testutil_test.go:52-59`) never sets `cfg.MaxInflight`, so it defaults to 0, and `maxInflightMiddleware(0)` (`rate_limit.go:18-20`) is a permanent no-op under `env.router` — **no other test in the suite can ever trigger a real 429 shed against the actual production wiring.**

Net effect: if a future change reorders `r.Use(...)` in `server.go` (e.g. `maxInflightMiddleware` before `securityHeadersMiddleware`), a real 429-shed response in production would silently lose all Tier-A headers, and **none of the 8 tests would catch it** — this is precisely the \$11.4.134 “would a paired mutation make a test FAIL?” check, and today the answer is no for this specific regression.

Fix: rebuild this test using `NewServer(Options{Config: config.Config{MaxInflight: 1, ...}}).Router()` (analogous to the existing `newSecHeadersRouter` helper), so the `shed-preserved-headers` property is proven against the real production chain, not a hand-authored copy of it.

I3 — Panic → recoveryMiddleware → 500 path is untested for Tier-A header presence, despite being an explicit claim in the code

Files: `server.go:118-121` comment · `security_headers_test.go` (no panic-path test exists)

I verified by reading the real `gin v1.12.0` source (`response_writer.go:66-81`) that this **currently holds correctly**: `responseWriter.WriteHeader` only records the intended status in memory and does not flush to the wire until the first `Write/Flush` call (`WriteHeaderNow`), and `securityHeadersMiddleware` sets its headers on the shared `http.Header` map well before any handler can panic — so `recoveryMiddleware`’s `respondError → c.AbortWithStatusJSON(500, ...)` correctly flushes the already-set Tier-A headers on a genuine panic today. This is **not a live bug** — but it is completely unverified by the test suite, despite the middleware-order comment in `server.go` explicitly claiming “present even on ... error/panic responses.” A future change to `recoveryMiddleware` (e.g. constructing a fresh writer instead of reusing `c.Writer`) would silently break this guarantee with zero test signal.

Fix: add `TestSecurityHeaders_TierA_OnPanic500` — a bespoke router with `securityHeadersMiddleware + recoveryMiddleware + a handler that panics` — mirroring the structure of the existing 429-shed test (and

subject to the same I2 caveat: prefer building it against the real `Server.Router()` chain if a panicking route can be added safely, or clearly document why it can't).

MINOR

M1 — `style-src 'self' 'unsafe-inline'` is a real (if currently justified and likely unavoidable) CSS-injection surface

File: `security_headers.go:54-59`

'unsafe-inline' for styles does not permit script execution, but it does allow CSS-based data-exfiltration/attribute-selector techniques (e.g. `input[value^="a"]{background:url(...)} keylogging-via-CSS`) *if* an attacker ever achieves any HTML injection elsewhere in the SPA. Given React's default JSX escaping this is low-likelihood today, and the justification (React 19 runtime `<style>` injection, static `go:embed` bundle can't emit per-response nonces) is genuine and correctly reasoned — this is not a bug to fix now, but worth an explicit tracked follow-up (nonce-based `style-src` becomes possible only if the `index.html` document is ever served dynamically instead of as a static embedded file).

M2 — `X-Request-Id` reflects client input unbounded, with no length/charset cap

File: `middleware.go:22-32` (adjacent to, not inside, `security_headers.go`, but directly relevant to the “header value derived from request input” question)

`requestIDMiddleware` echoes the client-supplied `X-Request-Id` header verbatim with no length cap and no charset allow-list. Real CRLF/header-splitting injection is not practically exploitable here (HTTP/1.1 header framing plus Go's `net/http` already prevent smuggling raw CRLF inside a single header value), so this is not a Critical/Important finding, but an oversized or garbage client-supplied value gets reflected into every response on that connection and potentially logged elsewhere — a minor amplification / log-hygiene gap. Recommend capping to e.g. 128 bytes and restricting to a safe charset (`[A-Za-z0-9_-]`), falling back to a freshly generated ID otherwise.

NIT

N1 — fullscreen=() in the deny-all Permissions-Policy

File: `security_headers.go:28-34`

Blocks the Fullscreen API entirely for the Manager UI — a UX consideration, not a security bug, worth a second look only if a dashboard feature (log viewers, topology diagrams) ever wants fullscreen.

Verified clean (explicitly checked, no finding)

- **Middleware ordering:** confirmed via the real gin v1.12.0 source that Tier-A headers survive the 429 shed, 404 NoRoute, 401 auth failure, and (per I3's reasoning) a genuine panic/500 — `securityHeadersMiddleware` runs before `maxInflightMiddleware/compressionMiddleware` in `server.go:122`, and `compressWriter` wraps rather than replaces the underlying `http.Header` map, so header content set before the wrap survives.
- **SPA client-route fallback correctness:** I initially suspected gin's `createStaticHandler` (`routergroup.go:216-237`) — which calls `c.Writer.WriteHeader(http.StatusNotFound)` before handing off to `NoRoute` — might commit a 404 status that a later `c.Data(200, ...)` couldn't override. Verified via `response_writer.go` that gin's `WriteHeader` is lazily buffered (only committed on first `Write`), so the later 200 from the SPA fallback correctly wins. All 8 tests pass (`go test ./internal/api/... -run 'TestSecurityHeaders|TestSPACSP'` — 8/8 PASS, verified live, not assumed).
- **CSP directive completeness:** `object-src 'none'`, `base-uri 'self'`, `form-action 'self'`, `frame-ancestors 'none'` are all present and correct; `default-src 'none'` on the API tier correctly needs no additional `object-src/etc.` (fetch-directive fallback), while `base-uri/frame-ancestors` are correctly set explicitly since `default-src` does not cover those non-fetch directives.
- **X-XSS-Protection:** correctly omitted (deprecated; modern guidance is to omit rather than set 1; `mode=block`, which has its own history of introducing vulnerabilities in older browsers).
- **Clear-Site-Data on logout:** not applicable yet — confirmed via `repo grep` that no `/auth/logout` endpoint exists at all. Flag as a

forward note: add `Clear-Site-Data` when a logout endpoint is introduced.

- **COEP require-corp omission:** explicitly and reasonably justified in the code comment (high breakage risk, no material benefit here); COOP same-origin is still set, which is the right partial-isolation trade-off.
 - **No duplicate/conflicting header sets:** `handlers_client.go:25` sets the same `Cache-Control: no-store` value Tier B already sets — redundant but not conflicting. Tier B's `apiCSP` and Tier C's `spaCSP` are scoped to disjoint route groups (confirmed by `TestSecurityHeaders_TierB_ScopedToAPI / TestSecurityHeaders_TierB_NotOnSPAAssets`), so no response ever carries both.
 - **Test value-assertion adequacy (the other 7 tests):** `assertTierA` compares exact string values (not mere presence), so a mutation flipping e.g. `X-Frame-Options` to `SAMEORIGIN` or dropping the HSTS TLS-gate would be caught. The `TestSPACSP_CompatibleWithRealBundle` test ties the CSP to the real built `index.html`'s actual asset references and inline-script absence — a genuine anti-bluff check, not a rubber stamp.
-

Summary for the conductor

- **Verdict:** GO-WITH-FIXES.
- **Critical (1):** C1 — `connect-src 'self'` CSP justification is false for the actual shipped SPA bundle (`api-client.ts` `baseUrl` hardcodes `http://localhost:8080`; confirmed in the real embedded JS bytes); breaks the Manager UI (or gets CSP-blocked) in any deployment not literally at `localhost:8080`. `server/internal/api/embed.go:6-16` (undocumented build step), `clients/ota-manager/src/lib/api-client.ts:14`, `server/internal/api/manager-dist/assets/index-*.js` (shipped artifact).
- **Important (3):** I1 — HSTS never fires behind a TLS-terminating reverse proxy (`security_headers.go:124-126`, no `X-Forwarded-Proto` override). I2 — the 429-shed header-survival test doesn't exercise the real `Server.Router()` chain, so a real ordering regression ships undetected (`security_headers_test.go:112-141`). I3 — the panic/500 header-survival path is correct today (verified against real gin source) but has zero test coverage despite being an explicit claim in the code (`server.go:118-121`).
- This determines whether a fix pass is dispatched before the headers work is considered done — recommend dispatching a fix

for C1 first (highest real-world impact: the Manager UI is non-functional outside dev today), then I1–I3.

Conductor disposition (2026-07-10, independent verification §11.4.2/§11.4.6/ §11.4.134)

I independently re-checked every finding against the live tree before dispositioning — the review is NOT rubber-stamped:

- **C1 — CONFIRMED REAL (Critical).** Verified: `clients/ota-manager/src/lib/api-client.ts:14` is `const BASE_URL = import.meta.env.VITE_API_BASE_URL ?? "http://localhost:8080";` and the literal `http://localhost:8080` IS present in the embedded bundle `server/internal/api/manager-dist/assets/index-C51iU0QH.js`; no build script sets `VITE_API_BASE_URL`. So the Tier-C CSP `connect-src 'self'` (landed 992bd497) blocks the Manager UI's API calls at any origin $\neq$ `http://localhost:8080`. **The CSP is correct-in-principle; the BUNDLE is the defect.** Correct fix (NOT loosening the CSP): change the source default to a same-origin/relative value (`?? ""` → axios uses relative URLs) AND rebuild + re-embed the bundle (tracked item K). **Disposition:** the source file lives in `clients/ota-manager`, owned by the in-flight C-impl stream (§11.4.119 — not editable now); routed to the C-impl integration / item-K re-embed window. **BLOCKING for any non-localhost deployment** — folds into the §11.4.185 QA gate; the CSP MUST NOT ship to a real (non-localhost) deployment until the bundle is rebuilt same-origin. Not silently loosened.
- **I1 — CONFIRMED REAL (Important), operator-config-gated.** HSTS gates on `TLSCertFile` & `TLSKeyFile`; behind a TLS-terminating reverse proxy (a likely prod topology) HSTS never fires. Correct fix requires trusting `X-Forwarded-Proto=https` — which MUST be gated on a **trusted-proxy config** (blindly trusting a client-spoofable header is itself a vuln). **Disposition:** operator decision on trusted-proxy configuration (a config addition, not a blind autonomous edit). Tracked.
- **I2 — NARROWED to Minor.** Re-verified: `newSecHeadersRouter` builds the REAL `srv.Router()`, so **6 of the 8 tests already exercise production middleware wiring** (Tier-A on normal/404/401, HSTS gating, Tier-B/C scoping) — a `server.go` reordering that drops/moves `securityHeaders` out of the production chain WOULD be caught by those. Only the `shed-carries-headers` property uses a

hand-built composition chain, because a deterministic 429 shed needs a blocking route the real router lacks. Residual risk = one specific reorder (securityHeaders after maxInflight) in the shed path only. **Disposition:** add a real-router deterministic-shed test when a blocking-route test seam is introduced; low residual risk given the 6 real-router tests. Tracked follow-up.

- **I3 — Minor.** The panic → recovery → 500 header path is (per SR) correct via gin's lazy WriteHeader buffering but lacks a test. A faithful test needs a panic-route hook in the real chain (adding one to production wiring is itself a change). **Disposition:** tracked server test-seam follow-up.

Net: review recorded with independent verification; the one Critical (C1) has a precise correct fix routed to the owning stream + flagged BLOCKING-before-non-localhost-deploy (never silently loosened, never fake-closed); I1 is an operator config decision; I2/I3 are minor/tracked. §11.4.134 satisfied: no finding dismissed, none fake-passed; the CSP does not ship to real deployment until C1's bundle fix lands (§11.4.185).